

LangGraph — The Complete One-Stop Guide

From absolute basics → advanced patterns → interview prep. Every concept here is tied to real, runnable code so you can revise fast and recall under pressure.

Table of Contents

1. What is LangGraph & Why It Exists
 2. LangChain vs LangGraph
 3. The Mental Model: Graph, State, Nodes, Edges
 4. State — The Heart of LangGraph
 5. Reducers — How State Updates Merge
 6. Nodes
 7. Edges: Normal, Conditional & the Router Pattern
 8. Pattern 1 — Sequential Pipeline
 9. Pattern 2 — Parallel Execution (Fan-out / Fan-in)
 10. Pattern 3 — Conditional Routing + RAG
 11. Pattern 4 — Tools, ToolNode & Agent Loops
 12. Human-in-the-Loop (HITL)
 13. Persistence: Checkpointers, thread_id & Memory
 14. Deploying with Streamlit
 15. Best Practices & Common Pitfalls
 16. Quick Reference Cheat Sheet
 17. Interview Questions & Answers
-

1. What is LangGraph & Why It Exists

LangGraph is a library (built by the LangChain team) for building **stateful, multi-step LLM applications** as **graphs**.

Instead of writing your AI logic as one long messy Python script with `while` loops, `if/else` branches, and manual state passing, you describe your app as:

- **Nodes** → units of work (usually a function that calls an LLM or a tool)
- **Edges** → the wiring that decides which node runs next
- **State** → a shared dictionary that flows through every node

Think of it as a **flowchart you can actually execute**, where an LLM sits inside the boxes.

Why not just use plain Python / LangChain chains?

Plain chains are great for **linear** flows (`A → B → C`). They fall apart the moment you need:

- **Loops** ("keep rewriting until the reviewer approves")
- **Branching** ("route to the fee-PDF vs the academic-PDF")
- **Parallelism** ("run 3 safety checks at once, then merge")
- **Pausing** ("wait for a human to approve before continuing")
- **Memory/persistence** ("resume this exact session tomorrow")

LangGraph gives you all of these as **first-class primitives**.

2. LangChain vs LangGraph

Problem	LangChain (plain)	LangGraph
Loops & re-evaluation	Manual <code>while</code> loops and condition handling make workflows messy	Native graph loops with conditional edges keep workflows clean and scalable
Multi-agent state	Shared memory between agents must be managed manually	Centralized shared state flows across all agents automatically
Long waiting tasks	Needs external DBs or schedulers to pause/resume	Built-in persistent & resumable workflows handle waiting naturally
Human-in-the-loop approval	Approval & branching logic become complex fast	Human decisions handled cleanly via graph transitions (<code>interrupt</code>)
Scalability	Complex systems become hard to maintain	Designed for scalable multi-agent orchestration

3. The Mental Model: Graph, State, Nodes, Edges

Every LangGraph program follows the **same 7 steps**. Memorize this skeleton — it never changes:

```
from typing import TypedDict
from langgraph.graph import StateGraph, START, END

# 1. Define the State (the shared data structure)
class MyState(TypedDict):
    input: str
    output: str

# 2. Define Nodes (functions that take state, return a partial update)
def my_node(state: MyState) -> dict:
    return {"output": state["input"].upper()}

# 3. Create the graph builder
builder = StateGraph(MyState)

# 4. Add nodes
builder.add_node("my_node", my_node)

# 5. Add edges (wiring)
builder.add_edge(START, "my_node")
builder.add_edge("my_node", END)

# 6. Compile
app = builder.compile()

# 7. Run
result = app.invoke({"input": "hello"})
print(result["output"])  # HELLO
```

Key vocabulary:

- `StateGraph(MyState)` — the **builder**. You add nodes/edges to it.
- `START` / `END` — special built-in virtual nodes marking entry and exit.
- `.compile()` — freezes the graph into a runnable `app`.
- `.invoke(initial_state)` — runs the graph once, returns the **final state**.

4. State — The Heart of LangGraph

State is the single dictionary that every node reads from and writes to. Getting state right is 80% of LangGraph.

Golden rule: a node **returns only the keys it wants to change**, not the whole state. LangGraph merges that partial update back into the global state for you.

There are **4 ways** to define state. You used all four in `states.py`.

4.1 `TypedDict` — the default, most common

Best for **compile-time type hints**. Lightweight, no runtime validation.

```

from typing import TypedDict

class State(TypedDict):
    topic: str
    summary: str
    score: int

```

Use this **90% of the time**. Simple and fast. It does **not** validate data at runtime — a negative `score` will pass silently.

4.2 Pydantic `BaseModel` — for runtime validation

Best when you need to **guarantee** data is valid while the graph runs.

```

from pydantic import BaseModel, field_validator

class StateModel(BaseModel):
    topic: str
    summary: str
    score: int

    @field_validator('score')
    def score_positive(cls, v):
        if v < 0:
            raise ValueError("Score must be a positive integer")
        return v

```

Validates types **and** custom rules at runtime (e.g. score can't be negative). Slightly heavier; overkill for simple graphs.

4.3 `@dataclass` — validation via `__post_init__`

A middle ground. Supports defaults and post-init checks.

```

from dataclasses import dataclass, field

@dataclass
class State:
    topic: str = ""
    summary: str = ""
    score: int = field(default=0)

    def __post_init__(self):
        if self.score < 0:
            raise ValueError("Score must be a positive integer")

```

Clean defaults with `field(default=...)`; runs validation after init.

4.4 `MessagesState` — the built-in for chat apps

LangGraph ships a prebuilt state that already contains a `messages` list with the right reducer. Extend it when building chatbots/agents.

```

from langgraph.graph import MessagesState

class State(MessagesState):
    # already has: messages: list
    topic: str
    summary: str
    score: int

```

Saves you from manually wiring the `messages` reducer. Great for agents.

Which one should you pick?

Type	Validation	Speed	Best for
<code>TypedDict</code>	compile-time hints only	fastest	Most graphs (default choice)
Pydantic	runtime	slower	Strict data guarantees
<code>@dataclass</code>	via <code>__post_init__</code>	fast	Defaults + light validation
<code>MessagesState</code>		fast	Chatbots / agents with <code>messages</code>

5. Reducers — How State Updates Merge

By default, when a node returns `{"key": value}`, LangGraph **overwrites** `state["key"]`.

But sometimes you want to **combine** instead of overwrite — e.g. *append* a message to a list, or *merge* two dictionaries. That's what a **reducer** does.

You attach a reducer using `Annotated[type, reducer_function]`.

5.1 The built-in `add_messages` reducer

The most common reducer. It **appends** new messages instead of replacing the whole list — essential for conversations.

```
from typing import TypedDict, Annotated
from langgraph.graph.message import add_messages

class State(TypedDict):
    topic: str
    # Without add_messages, each node would ERASE the history.
    # With it, new messages are appended.
    messages: Annotated[list, add_messages]
```

5.2 Writing your own reducer (from `parallel.py`)

When 3 parallel nodes all write to the same `safety_scores` dict, you don't want the last one to win — you want them **merged**. So you write a custom reducer:

```
from typing import TypedDict, Annotated

def merge_score_dicts(existing: dict, newupdate: dict) -> dict:
    if existing is None:
        return newupdate
    # merge old + new. e.g. {"toxicity": 80} + {"copyright": 70}
    #                               => {"toxicity": 80, "copyright": 70}
    return {**existing, **newupdate}

class AnalyzerState(TypedDict):
    raw_text: str
    # "When multiple nodes update safety_scores, use merge_score_dicts to combine them"
    safety_scores: Annotated[dict[str, int], merge_score_dicts]
```

A reducer has the signature `reducer(existing_value, new_value) -> merged_value`. LangGraph calls it every time a node returns that key.

Why reducers matter for parallelism: if two nodes run at the same time and both write the same key **without** a reducer, LangGraph raises a concurrency error (`InvalidUpdateError`). The reducer tells LangGraph how to safely combine the two writes.

6. Nodes

A **node** is just a Python function:

- **Input:** the current `state`
- **Output:** a `dict` with **only the keys it wants to update**

```
def editor_node(state: PipelineState) -> dict:
    prompt = (
        "You are an expert copyeditor. Clean up the following raw text. "
        f"Text:\n{state['raw_input']}"
    )
    response = llm.invoke(prompt)
    return {"edited_text": response.content.strip()} # only updates edited_text
```

Rules of a good node:

1. **Pure-ish:** read from `state`, return a partial `dict`. Don't mutate `state` in place.
2. **Return only what changed.** Returning `{"edited_text": ...}` leaves every other key untouched.
3. **Name it clearly** when adding: `builder.add_node("Editor Node", editor_node)`.
4. A node can call an LLM, a tool, a retriever, do plain Python — anything.

Special node: `ToolNode`

LangGraph provides a prebuilt node that executes tool calls automatically (covered in [Section 11](#)):

```
from langgraph.prebuilt import ToolNode
tool_node = ToolNode(tools) # runs whatever tool the LLM asked for
```

7. Edges: Normal, Conditional & the Router Pattern

Edges decide *what runs next*. There are two kinds.

7.1 Normal (static) edges

Always go from A → B. Unconditional.

```
graph.add_edge(START, "Editor Node")
graph.add_edge("Editor Node", "Scriptwriter Node")
graph.add_edge("Scriptwriter Node", END)
```

7.2 Conditional edges (the router pattern)

Go to **different** nodes based on logic. You provide a **router function** that returns the **name of the next node** (a string).

```
# The router: looks at state, returns the name of the next node
def route_query(state: State):
    if state['query_type'] == 'academic':
        return "academic_rag"
    elif state['query_type'] == "fee":
        return "fee_rag"
    else:
        return "general"

# Wire it: after "classifier", call route_query to pick the branch
graph.add_conditional_edges("classifier", route_query)
```

Optional mapping argument. You can pass an explicit `{return_value: node_name}` map, which also documents the possible destinations:

```
graph.add_conditional_edges(
    "human_review",
    should_stop_looping,
    {
        "writer": "writer",    # if router returns "writer" → go to writer node
        END: END,              # if router returns END → finish
    },
)
```

A router function **returns a string (or END)** — it does *not* return state. That string must match a node name (or a key in the optional mapping).

8. Pattern 1 — Sequential Pipeline

Use case: a fixed assembly line where each stage transforms the output of the previous one. **Your example (`project.py`):** raw text → **Editor** → **Scriptwriter** → **Translator** → Hinglish output.

```
START → Editor → Scriptwriter → Translator → END
```

The state carries data between stages

```
from typing import TypedDict

class PipelineState(TypedDict):
    raw_input: str
    edited_text: str
    script_text: str
    final_output: str
```

Each node reads the previous key, writes the next

```
def editor_node(state: PipelineState) -> dict:
    prompt = f"Clean up this text. Return only edited text.\n\n{state['raw_input']}"
    return {"edited_text": llm.invoke(prompt).content.strip()}

def scriptwriter_node(state: PipelineState) -> dict:
    prompt = f"Turn this into a punchy video script.\n\n{state['edited_text']}"
    return {"script_text": llm.invoke(prompt).content.strip()}

def translator_node(state: PipelineState) -> dict:
    prompt = f"Convert this script into natural Hinglish.\n\n{state['script_text']}"
    return {"final_output": llm.invoke(prompt).content.strip()}
```

Wiring: a straight line

```
graph = StateGraph(PipelineState)
graph.add_node("Editor Node", editor_node)
graph.add_node("Scriptwriter Node", scriptwriter_node)
graph.add_node("Translator Node", translator_node)

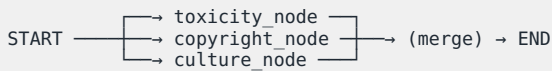
graph.add_edge(START, "Editor Node")
graph.add_edge("Editor Node", "Scriptwriter Node")
graph.add_edge("Scriptwriter Node", "Translator Node")
graph.add_edge("Translator Node", END)

app = graph.compile()
result = app.invoke({"raw_input": "AI agents are the future of tech..."})
print(result["final_output"])
```

Takeaway: sequential = one `add_edge` per hop. Data flows forward through distinct state keys.

9. Pattern 2 — Parallel Execution (Fan-out / Fan-in)

Use case: run several **independent** analyses at the same time, then combine results. **Your example (`parallel.py`):** analyze one script for **toxicity**, **copyright**, and **cultural sensitivity** simultaneously.



Fan-out: multiple edges from START

```
builder.add_edge(START, "toxicity_node")
builder.add_edge(START, "copyright_node")
builder.add_edge(START, "culture_node")
```

Because all three start from `START`, LangGraph runs them **in parallel**.

Fan-in: they all write the same key → needs a reducer

All three nodes update `safety_scores`. Without a reducer this crashes. With `merge_score_dicts`, the results merge cleanly:

```
class AnalyzerState(TypedDict):
    raw_text: str
    safety_scores: Annotated[dict[str, int], merge_score_dicts]

def toxicity_node(state):
    # ... ask LLM for a 0-100 score ...
    return {"safety_scores": {"toxicity_level": score}}

def copyright_node(state):
    return {"safety_scores": {"copyright_risk": score}}

def culture_node(state):
    return {"safety_scores": {"cultural_insensitivity": score}}
```

Each branch ends independently:

```
builder.add_edge("toxicity_node", END)
builder.add_edge("copyright_node", END)
builder.add_edge("culture_node", END)
```

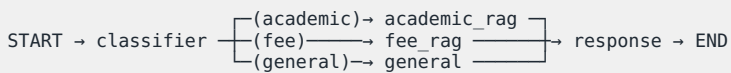
Final merged output:

```
{
  "toxicity_level": 95,
  "copyright_risk": 80,
  "cultural_insensitivity": 60
}
```

Parallel = many edges out of one node + a reducer on the shared key. The reducer is *mandatory* here — it's what makes concurrent writes safe.

10. Pattern 3 — Conditional Routing + RAG

Use case: classify the user's intent, then send them down the right branch. All branches converge on one response node. **Your example (`conditional.py` / `app.py`):** a college assistant that routes to **Academic PDF (RAG)**, **Fee PDF (RAG)**, or **General knowledge**.



Step 1 — Build RAG retrievers from PDFs

```
from langchain_community.document_loaders import PyPDFLoader
from langchain_text_splitters import RecursiveCharacterTextSplitter
from langchain_huggingface import HuggingFaceEmbeddings
from langchain_community.vectorstores import FAISS

embeddings = HuggingFaceEmbeddings(model_name="sentence-transformers/all-MiniLM-L6-v2")

def build_retriever(pdf_path: str):
    loader = PyPDFLoader(pdf_path)
    documents = loader.load()
    splitter = RecursiveCharacterTextSplitter(chunk_size=800, chunk_overlap=100)
    chunks = splitter.split_documents(documents)
    vectorstore = FAISS.from_documents(chunks, embeddings)
    return vectorstore.as_retriever(search_kwargs={"k": 4})

academic_retriever = build_retriever("academics_handbook.pdf")
fee_retriever = build_retriever("fee_structure.pdf")
```

RAG in one sentence: load PDF → split into chunks → embed chunks into vectors → store in FAISS → retrieve the top-k most similar chunks for a query → feed them to the LLM as context.

Step 2 — Classifier node decides the category

```
def classifier_node(state: State) -> dict:
    last_message = state["messages"][-1].content
    prompt = (
        "Classify into exactly one category: academic, fee, or general.\n"
        f"Query: {last_message}\nReturn ONLY one word."
    )
    category = llm.invoke(prompt).content.strip().lower()
    if "academic" in category: category = "academic"
    elif "fee" in category: category = "fee"
    else: category = "general"
    return {"query_type": category}
```

Step 3 — Router + conditional edges

```
def route_query(state: State):
    if state["query_type"] == "academic": return "academic_rag"
    elif state["query_type"] == "fee": return "fee_rag"
    else: return "general"

graph.add_edge(START, "classifier")
graph.add_conditional_edges("classifier", route_query)

# All three branches converge on "response"
graph.add_edge("academic_rag", "response")
graph.add_edge("fee_rag", "response")
graph.add_edge("general", "response")
graph.add_edge("response", END)
```

Step 4 — Response node personalizes using retrieved context

```
def response_node(state: State) -> dict:
    query = state["messages"][-1].content
    programme = state.get("programme", "Unknown")
    context = state["retrieved_context"]

    if context == "NO RETRIEVAL NEEDED":
        prompt = f"You are a friendly assistant for a {programme} student.\n{query}"
    else:
        prompt = (
            f"Use this context to answer for a {programme} student.\n"
            f"Context:\n{context}\n\nQuestion: {query}"
        )
    return {"messages": [("ai", llm.invoke(prompt).content.strip())]}
```

Convergence: notice all branches point to `response`. This is the "every conditional path converges to a single node" idea — clean and DRY.

11. Pattern 4 — Tools, ToolNode & Agent Loops

Use case: give the LLM tools (web search, calculators, APIs) and let it decide when to use them — then loop until the work is approved. **Your example (`interactive_tools.py`):** a **Writer** that can web-search, an **Extractor**, and a strict **Reviewer** that loops back until the post is APPROVED (max 3 attempts).

```
START → writer —(tool_calls?)→ tools → writer (loop for search)
      | (no tool)→ extract_draft → reviewer
      | (rejected & <3)
      | → writer (rewrite)
      | (approved OR 3 attempts) → END
```

Step 1 — Define tools and bind them to the LLM

```
from langchain_tavily import TavilySearch
from langchain_groq import ChatGroq

search_tool = TavilySearch(max_results=3, api_key=os.environ.get("TAVILY_API_KEY"))
tools = [search_tool]

writer_llm = ChatGroq(model="openai/gpt-oss-120b", temperature=0.7)
writer_llm_with_tools = writer_llm.bind_tools(tools) # LLM can now REQUEST tools
```

`bind_tools` doesn't *run* tools — it lets the LLM emit **tool-call requests** in its response.

Step 2 — State uses `add_messages` to accumulate the conversation

```
class State(TypedDict):
    topic: str
    messages: Annotated[list, add_messages] # appends, never overwrites
    draft: str
    review_feedback: str
    is_approved: bool
    attempt: int
```

Step 3 — ToolNode executes whatever the LLM requested

```
from langgraph.prebuilt import ToolNode
tool_node = ToolNode(tools)
```

Step 4 — Router: did the LLM ask for a tool?

```
def should_use_tool(state: State):
    last_message = state["messages"][-1]
    if getattr(last_message, "tool_calls", None): # LLM requested a tool
        return "tools"
    return "extract_draft" # no tool → move on
```

Step 5 — Router: should we keep looping?

```
def should_stop_looping(state: State):
    if state["is_approved"]:
        return END # approved → done
    if state["attempt"] >= 3:
        return END # safety cap → done
    return "writer" # rejected → rewrite
```

Always add a loop guard (`attempt >= 3`). Without a max-attempt cap, a picky reviewer + stubborn writer can loop forever and burn tokens.

Step 6 — Wire the graph (note the cycle!)

```
graph.add_edge(START, "writer")
graph.add_conditional_edges("writer", should_use_tool) # writer → tools OR extract_draft
graph.add_edge("tools", "writer") # tool result goes BACK to writer to read it
graph.add_edge("extract_draft", "reviewer")
graph.add_conditional_edges("reviewer", should_stop_looping) # reviewer → writer OR END
```

The two loops that make this an "agent":

1. **Tool loop:** `writer → tools → writer` — the writer sees search results and continues.
2. **Review loop:** `reviewer → writer → ... → reviewer` — rewrite until approved.

This *cyclic* graph is exactly what LangGraph does that plain chains cannot.

12. Human-in-the-Loop (HITL)

Use case: pause the graph, show a draft to a **human**, and resume only after they approve or give feedback. **Your example** (`humanintheLoop.py`): the Reviewer is replaced by a **real person** typing in the terminal.

The 4 primitives (memorize these)

Primitive	Role	Memory hook
<code>interrupt(payload)</code>	Pauses the graph, sends <code>payload</code> out to the human	<code>interrupt = PAUSE</code>
<code>Command(resume=value)</code>	Resumes the graph; <code>value</code> becomes the return of <code>interrupt()</code>	<code>resume = CONTINUE</code>
Checkpoint (<code>MemorySaver</code>)	Saves state so pause/resume works	<code>checkpointer = SAVE</code>
<code>thread_id</code>	Identifies the session to resume	<code>thread_id = IDENTIFY SESSION</code>

```
from langgraph.types import interrupt, Command

def human_review_node(state: State) -> dict:
    # Execution STOPS here. The dict is handed to the outside world.
    human_response = interrupt({
        "draft": state["draft"],
        "attempt": state["attempt"],
        "instruction": "Type 'approved' to accept, or type feedback to request a rewrite."
    })
    # This code only runs AFTER resume. human_response == the resumed value.
    response = human_response.strip()
    if response.lower() in ["approved", "approve", "yes", "ok", "good"]:
        return {"is_approved": True, "review_feedback": "Approved by human."}
    return {"is_approved": False, "review_feedback": response}
```

Step 2 — Compile WITH a checkpointer (required for HITL)

```
from langgraph.checkpoint.memory import MemorySaver

checkpointer = MemorySaver()
app = graph.compile(checkpointer=checkpointer) # no checkpointer = no pause/resume
```

Step 3 — Run, detect the interrupt, resume in a loop

```
config = {"configurable": {"thread_id": "linkedin_session_1"}} # identifies session
result = app.invoke(initial_state, config=config)

# Keep resuming while the graph is paused
while "__interrupt__" in result:
    interrupt_data = result["__interrupt__"][0].value # what the node sent out
    print(interrupt_data["draft"])

    human_input = input("Your response: ").strip()

    # Resume the SAME thread with the human's answer
    result = app.invoke(Command(resume=human_input), config=config)

print("FINAL:", result["draft"])
```

The core HITL flow

Node → `interrupt()` → « HUMAN types → `Command(resume=input)` → Graph resumes

Two non-negotiables for HITL: (1) compile with a **checkpointer**, and (2) pass the same `thread_id` every time so LangGraph knows which paused session to continue.

13. Persistence: Checkpointers, thread_id & Memory

A **checkpointer** saves a snapshot of the state after every step. This unlocks three superpowers:

1. **Pause / resume** (HITL, long-running tasks)
2. **Memory across turns** — the same `thread_id` remembers previous messages
3. **Time-travel / recovery** — inspect or replay past states

The most common checkpointer

```
from langgraph.checkpoint.memory import MemorySaver
checkpointer = MemorySaver() # in-RAM, great for dev/demos (lost on restart)
app = graph.compile(checkpointer=checkpointer)
```

For production you'd swap in a persistent one (e.g. `SQLiteSaver`, `PostgresSaver`) so state survives restarts — but the API is identical.

thread_id = one conversation/session

```
config = {"configurable": {"thread_id": "user_42"}}
app.invoke(state, config=config) # every call with this id shares memory
```

- **Same** `thread_id` → continues the same conversation (state is remembered).
- **Different** `thread_id` → a fresh, isolated session.

Mental model: `checkpointer` = the notebook that saves everything; `thread_id` = the name on the notebook's cover that tells you *which* conversation to open.

14. Deploying with Streamlit

Your example (`app.py`): the college RAG assistant wrapped in a chat UI.

Two Streamlit ideas make LangGraph apps production-friendly:

14.1 Cache expensive setup with `@st.cache_resource`

Building embeddings + FAISS retrievers is slow. Cache them so they run **once**, not on every rerun:

```
@st.cache_resource(show_spinner="Loading knowledge base...")
def load_resources():
    embeddings = HuggingFaceEmbeddings(model_name="sentence-transformers/all-MiniLM-L6-v2")
    academic_retriever = build_retriever("academics_handbook.pdf")
    fee_retriever = build_retriever("fee_structure.pdf")
    llm = ChatGroq(model="openai/gpt-oss-120b", temperature=0.4)
    return academic_retriever, fee_retriever, llm

@st.cache_resource(show_spinner=False)
def build_graph():
    # ... add nodes/edges ...
    return graph.compile()
```

Streamlit reruns your whole script on **every** interaction. Without `@st.cache_resource`, you'd rebuild the vector store on every keystroke — slow and expensive.

14.2 Persist chat history in `st.session_state`

```
if "lc_messages" not in st.session_state:
    st.session_state.lc_messages = [] # LangGraph message history

if user_query := st.chat_input("Type your question..."):
    st.session_state.lc_messages.append(("human", user_query))
    result = app.invoke({
        "programme": student_programme,
        "messages": st.session_state.lc_messages,
    })
    st.session_state.lc_messages = result["messages"] # save updated history
```

Takeaway: `@st.cache_resource` for heavy objects (models, graphs, retrievers); `st.session_state` for per-user conversation state.

15. Best Practices & Common Pitfalls

Do

- **Return partial updates** from nodes — only the keys that changed.
- **Use** `add_messages` for any conversational `messages` list.
- **Add a loop guard** (`attempt >= N`) to every cycle to prevent infinite loops.
- **Use a reducer** whenever multiple nodes write the same key (especially in parallel).
- **Cache heavy resources** (retrievers, models, compiled graph) in deployment.
- **Give nodes clear, descriptive names** — they show up in traces and errors.
- **Compile with a checkpointer** for anything needing memory or HITL.

Avoid / Watch out

Pitfall	Fix
Two parallel nodes write the same key with no reducer → <code>InvalidUpdateError</code>	Add <code>Annotated[type, reducer]</code>
Node mutates <code>state</code> in place and returns nothing	Return a <code>dict</code> of changed keys
Infinite loop in a review/agent cycle	Add a max-attempt guard in the router
Router returns a name that isn't a node	Return exact node names (or use the mapping dict)
HITL "doesn't resume"	You forgot the checkpointer or changed the thread_id
Streamlit rebuilds vector store every keystroke	Wrap setup in <code>@st.cache_resource</code>
Overwriting chat history each turn	Use <code>add_messages</code> reducer

16. Quick Reference Cheat Sheet

```
# ----- IMPORTS -----
from typing import TypedDict, Annotated
from langgraph.graph import StateGraph, START, END
from langgraph.graph.message import add_messages
from langgraph.prebuilt import ToolNode
from langgraph.checkpoint.memory import MemorySaver
from langgraph.types import interrupt, Command

# ----- STATE -----
class State(TypedDict):
    messages: Annotated[list, add_messages] # append-only list
    data: str # overwrite-by-default

# ----- NODE -----
def my_node(state: State) -> dict:
    return {"data": "updated"} # only changed keys

# ----- BUILD -----
g = StateGraph(State)
g.add_node("my_node", my_node)

# ----- EDGES -----
g.add_edge(START, "my_node") # static edge
g.add_edge("my_node", END)
g.add_conditional_edges("my_node", router_fn) # dynamic edge
g.add_conditional_edges("n", router_fn, {"a": "a", END: END}) # with mapping

# ----- COMPILE & RUN -----
app = g.compile() # or compile(checkpointer=MemorySaver())
result = app.invoke({"messages": []})
# with memory/HITL:
cfg = {"configurable": {"thread_id": "s1"}}
result = app.invoke(state, config=cfg)

# ----- HITL -----
val = interrupt({"ask": "approve?"}) # pause
app.invoke(Command(resume="yes"), config=cfg) # resume
```

Concept	API
Create graph	<code>StateGraph(State)</code>
Add node	<code>.add_node("name", fn)</code>
Static edge	<code>.add_edge("a", "b")</code>
Dynamic edge	<code>.add_conditional_edges("a", router)</code>
Entry / exit	<code>START</code> , <code>END</code>
Compile	<code>.compile()</code>
Run	<code>.invoke(state, config=...)</code>
Append messages	<code>Annotated[list, add_messages]</code>
Custom merge	<code>Annotated[T, my_reducer]</code>
Run tools	<code>ToolNode(tools)</code>
Bind tools to LLM	<code>llm.bind_tools(tools)</code>
Pause	<code>interrupt(payload)</code>
Resume	<code>Command(resume=value)</code>
Save state	<code>compile(checkpointer=MemorySaver())</code>
Session id	<code>{"configurable": {"thread_id": "..."} }</code>

17. Interview Questions & Answers

Basics

Q1. What is LangGraph and how does it differ from LangChain? LangGraph is a library for building **stateful, graph-based** LLM applications. LangChain provides the *components* (LLMs, retrievers, tools, chains); LangGraph provides the *orchestration* — a state machine with nodes and edges that supports loops, branching, parallelism, persistence, and human-in-the-loop, which plain LangChain chains handle awkwardly.

Q2. What are the core building blocks of a LangGraph app? **State** (shared data dict), **Nodes** (functions that do work), **Edges** (wiring that decides the next node), and the special **START/END** markers. You build with `StateGraph`, then `.compile()` into a runnable app.

Q3. What is "State" and what does a node return? State is the shared data structure that flows through the graph. A node receives the current state and returns a **partial dict** containing only the keys it wants to update — LangGraph merges that back into the global state. Nodes should not mutate state in place.

Q4. What are the ways to define state? When would you use each? `TypedDict` (default, compile-time hints, fastest), `Pydantic BaseModel` (runtime validation with `@field_validator`), `@dataclass` (defaults + `__post_init__` validation), and `MessagesState` (prebuilt state with a `messages` list + reducer, ideal for chatbots).

Q5. What do START and END represent? They are special virtual nodes. `START` marks where execution begins; `END` marks where it stops. You connect real nodes to them with edges (`add_edge(START, "first")`, `add_edge("last", END)`).

Q6. What's the difference between .invoke() and compiling? `.compile()` turns the graph *builder* into an executable *app* (validates structure, wires reducers). `.invoke(initial_state)` actually runs the compiled graph once and returns the final state.

Intermediate

Q7. What is a reducer? Give an example. A reducer defines **how a state key's updates are combined** instead of overwritten. You attach it with `Annotated[type, reducer_fn]`. Example: `add_messages` appends new messages to the list. A custom `merge_score_dicts(existing, new)` merges two dicts so parallel nodes can safely write the same key.

Q8. What is add_messages and why is it important? It's the built-in reducer for message lists. Without it, each node returning `messages` would **replace** the entire history; with it, new messages are **appended**, preserving the conversation. Essential for chat and agent graphs.

Q9. How do static edges differ from conditional edges? A static edge (`add_edge("a", "b")`) always goes A→B. A conditional edge (`add_conditional_edges("a", router)`) calls a **router function** that inspects state and returns the **name of the next node** (or `END`), enabling branching.

Q10. What does a router function return? A **string** matching a node name (or `END`), *not* a state dict. Optionally you pass a mapping `{return_value: node_name}` as the third argument to make destinations explicit.

Q11. How do you implement parallel execution? Add multiple edges out of a single node (e.g. all from `START`). Those nodes run concurrently. Because they typically write the same key, attach a **reducer** to that key so concurrent writes merge safely instead of raising `InvalidUpdateError`.

Q12. Walk through a conditional-routing RAG pipeline. A `classifier` node labels the query (academic/fee/general). `add_conditional_edges` routes to the matching branch: academic/fee branches retrieve chunks from their FAISS vector store; general skips retrieval. All branches converge on a single `response` node that builds the final answer from the retrieved context. → `START → classifier → {branch} → response → END`.

Q13. Explain the RAG steps used inside a node. Load PDF (`PyPDFLoader`) → split into chunks (`RecursiveCharacterTextSplitter`) → embed (`HuggingFaceEmbeddings`) → store in a vector DB (`FAISS`) → `retriever.invoke(query)` returns top-k similar chunks → pass them as context to the LLM.

Q14. How do you build an agent that uses tools? `llm.bind_tools(tools)` lets the LLM emit tool-call requests. A router checks `last_message.tool_calls`: if present → route to a `ToolNode(tools)` which executes them, then edge back to the LLM so it can read results; if absent → continue. This creates the `llm → tools → llm` loop.

Advanced

Q15. How do you prevent infinite loops in cyclic graphs? Track an attempt counter in state and guard it in the router: `if state["attempt"] >= 3: return END`. Always cap review/agent loops so a stubborn cycle can't run forever and burn tokens. (LangGraph also has a global `recursion_limit` as a backstop.)

Q16. Explain Human-in-the-Loop in LangGraph. Inside a node, call `interrupt(payload)` to **pause** execution and surface `payload` to the outside world. The app detects `"__interrupt__"` in the result, collects human input, then calls `app.invoke(Command(resume=value), config=config)` to **resume** — `value` becomes the return of `interrupt()`. Requires a **checkpoint** and a stable `thread_id`.

Q17. Why is a checkpoint required for HITL? Pausing means the graph must persist its state and later restore it exactly. The checkpoint (e.g. `MemorySaver`) saves a snapshot after each step; on resume, LangGraph loads the snapshot for that `thread_id` and continues from the interrupt. No checkpoint → nothing to resume from.

Q18. What is `thread_id` and how does it enable memory? `thread_id` (inside `config["configurable"]`) names a session. The checkpoint stores state per thread, so re-invoking with the **same** `thread_id` continues the same conversation (remembers history); a **new** `thread_id` starts fresh. It's how one compiled app serves many isolated users.

Q19. How does LangGraph detect and read an interrupt in the outer loop? Check `"__interrupt__" in result`. The payload sent by the node is at `result["__interrupt__"][0].value`. Loop: while interrupted, show the payload, get input, `invoke(Command(resume=input), config)`.

Q20. What happens if two concurrent nodes update the same key without a reducer? LangGraph raises an `InvalidUpdateError` (concurrent conflicting writes). The fix is a reducer via `Annotated[type, reducer]` telling LangGraph how to merge the two updates.

Q21. In deployment, why cache the graph/retrievers, and how? UI frameworks like Streamlit rerun the whole script on every interaction. Rebuilding embeddings/FAISS/the compiled graph each time is slow and costly. Wrap that setup in `@st.cache_resource` so it initializes once and is reused across reruns; keep per-user conversation in `st.session_state`.

Q22. When would you choose Pydantic state over TypedDict? When you need **runtime guarantees** — validating types and business rules (e.g. `score ≥ 0`) while the graph runs, failing fast on bad data. `TypedDict` only gives static hints and won't catch a bad value at runtime.

Q23. Sequential vs parallel vs conditional — one line each. Sequential: one edge per hop, data flows forward (`A→B→C`).

Parallel: many edges from one node + a reducer to merge (fan-out/fan-in). **Conditional:** a router picks the next node from state, branches usually converge on one node.

Q24. What makes LangGraph "cyclic" and why does that matter? Unlike a DAG-only chain, LangGraph edges can point **backward** (e.g. `reviewer → writer`), forming cycles. This enables iterative refinement, retries, and agent loops — the defining capability that plain sequential chains lack.

Final 30-second recap

State flows through **nodes**, connected by **edges**. Static edges go one way; **conditional edges** branch via a **router**. **Reducers** (`add_messages`, custom) control how updates merge — mandatory for **parallel** writes. Cycles enable **loops/agents** (guard them!). **Checkpoint + thread_id** unlock **memory** and **human-in-the-loop** (`interrupt → Command(resume=...)`). Cache heavy setup in deployment.